

Universitatea Alexandru Ioan Cuza
Facultatea de Informatica

Sistem de Telemedicină

Practică SGBD

Student: Mătăsel Sara Giulia

Grupă: 2A2

Cuprins

1	Introducere	3
2	Cerințe de business	3
2.1	Gestionarea pacienților și tutorilor	3
2.2	Abonamente și control acces	4
2.3	Evaluare medicală automată	4
2.4	Programarea consultațiilor	4
2.5	Consultația cu medicul	5
2.6	Istoric medical	5
3	Normalizarea bazei de date	5
3.1	Relația universală	5
3.2	Dependențe funcționale	6
3.3	Dependențe multivaluate	6
3.4	Găsirea cheilor candidat	7
3.5	Forma Normală 1 (1NF)	7
3.6	Forma Normală 2 (2NF)	8
3.7	Forma Normală 3 (3NF)	9
3.8	BCNF (Boyce-Codd Normal Form)	10
3.9	Forma Normală 4 (4NF)	11
3.10	Concluzie	12
4	Diagrama Entitate-Asociere	13
4.1	Entități și asocieri	13
4.2	Chei primare și decizii de design	14
4.3	Constrângeri implementate	14
5	Scriptul de creare a tabelor	15
5.1	Secțiunile DROP și secvențe	15
5.2	Tabela PATIENT	15
5.3	Tabela DOCTOR	16
5.4	Tabela SUBSCRIPTION	16
5.5	Tabela MEDICAL_FORM	17
5.6	Tabela CONSULTATION	17
5.7	Tabela FORM_ANSWER și indecși	18
6	Scriptul de populare automată	18
6.1	Structura generală	18
6.2	Generarea pacienților	18
6.3	Generarea pacienților minori cu tutori	19
6.4	Generarea abonamentelor cu status corect	19
6.5	Instrucțiunea RETURNING	19
7	Prezentarea aplicației	20
7.1	Arhitectura aplicației	20
7.2	Autentificare și înregistrare	20
7.3	Dashboard pacient	21

7.4	Fluxul de consultatie	22
7.5	Algoritmul de scheduling	24
7.6	Triggere	25
7.7	Exceptii custom prinse în client	26
7.8	Dashboard doctor și rapoarte	26
7.9	Istoricul pacientului	28
7.10	Funcționalități avansate implementate	29

1. Introducere

Lucrarea prezintă proiectarea și implementarea unui sistem de telemedicină destinat managementului virtual al consultațiilor medicale. Aplicația permite înregistrarea și administrarea pacienților, gestionarea abonamentelor medicale, evaluarea automată a simptomelor și programarea consultațiilor online cu medici specializați.

Sistemul a fost dezvoltat folosind **PostgreSQL** ca server de baze de date, cu logică de business implementată la nivel de server prin proceduri și funcții **PL/pgSQL**, și o aplicație client de tip web scrisă în **Java Spring Boot** care apelează aceste proceduri prin **JDBC** pur, fără utilizarea niciunui ORM.

Sistemul implementează următoarele fluxuri principale:

- Înregistrare și autentificare pacienți și medici
- Gestionare abonamente cu alerte automate de expirare
- Generare automată de fișe medicale pe baza simptomelor, vârstei și afecțiunilor cronice
- Programare automată a consultațiilor cu algoritm de scheduling
- Rețetă automată pentru cazuri simple
- Portal medic cu confirmare diagnostic și prescriere rețetă
- Rapoarte și statistici

2. Cerințe de business

Pe baza scenariului descris, am identificat următoarele cerințe de business:

2.1 Gestionarea pacienților și tutorilor

- CB1** Fiecare pacient este înregistrat cu date de identificare (nume, prenume, dată naștere), date de contact (email unic, număr de telefon) și adresă.
- CB2** Vârsta pacientului se calculează automat din data nașterii și este utilizată în procesul de evaluare medicală. Sistemul clasifică pacientul în categorii de vârstă: sugar (sub 2 ani), copil (2–11 ani), adolescent (12–17 ani), adult (18–64 ani), vârstnic (peste 65 ani).
- CB3** Un pacient minor poate fi reprezentat de un tutore, care este el însuși un pacient înregistrat în sistem. Relația este modelată printr-o cheie străină reflexivă în tabela **PATIENT**.
- CB4** Un tutore poate reprezenta mai mulți copii, rezultând o relație de tip 1:N între tutore și pacienții minori.
- CB5** Pacienții se pot înregistra în platformă prin intermediul unei pagini de înregistrare, cu validare pe partea de client și server.

2.2 Abonamente și control acces

- CB6** Accesul la serviciile de telemedicină este condiționat de existența unui abonament activ. Această regulă este verificată la nivel de server de baze de date, prin excepție custom cu codul P0001.
- CB7** Un abonament poate fi de tip lunar (49,99 RON) sau anual (499,99 RON).
- CB8** Pentru fiecare abonament se păstrează istoricul complet al plăților, inclusiv metoda de plată și data tranzacției.
- CB9** Statusul abonamentului (*activ/expirat*) este determinat automat pe baza datei de expirare printr-un trigger.
- CB10** Sistemul generează alerte pentru pacienții al căror abonament expiră în următoarele 7 zile, vizibile în dashboard-ul pacientului și în rapoartele medicului.

2.3 Evaluare medicală automată

- CB11** Inițierea unei evaluări medicale presupune introducerea a exact trei simptome primare de către pacient.
- CB12** Pe baza simptomelor introduse, a vârstei pacientului, a categoriei de vârstă și a afecțiunilor cronice cunoscute, sistemul generează automat o fișă medicală cu un diagnostic provizoriu și întrebări suplimentare adaptate contextului.
- CB13** Fiecărei fișe medicale i se atribuie un nivel de complexitate între 1 și 3, care determină pașii următori în flux.
- CB14** Cazurile de urgență (complexitate 3 cu simptome critice precum dureri abdominale acute și vărsături) sunt semnalate explicit cu un banner roșu și directionate prioritar.
- CB15** Pacientul răspunde la întrebările generate automat, răspunsurile fiind salvate în tabela FORM_ANSWER.
- CB16** La o evaluare nouă, sistemul detectă automat simptome similare cu istoricul pacientului și afiează un avertisment relevant.

2.4 Programarea consultațiilor

- CB17** Cazurile de complexitate 1 cu diagnostic simplu primesc o rețetă automată fără antibiotice, cu medicamente disponibile fără prescripție medicală.
- CB18** Cazurile de complexitate 2 și 3 sunt programate automat la cel mai apropiat slot disponibil al unui medic.
- CB19** Durata consultației este de 10 minute pentru complexitate 1, 20 de minute pentru complexitate 2 și 30 de minute pentru complexitate 3.
- CB20** Algoritmul de programare ține cont de programul zilnic al fiecărui doctor, de sloturile deja ocupate și de nivelul de complexitate.
- CB21** Dacă nu există niciun slot disponibil în următoarele 14 zile, pacientul este adăugat automat într-o listă de așteptare.

2.5 Consultația cu medicul

- CB22** În urma consultației, doctorul confirmă sau modifică diagnosticul provizoriu și poate prescrie o rețetă.
- CB23** Doctorul poate decide trimiterea pacientului la spital, către investigații suplimentare sau către un specialist, decizie înregistrată în sistem.
- CB24** Orice rețetă emisă este salvată cu data emiterii, diagnosticul asociat și lista medicamentelor recomandate.

2.6 Istoric medical

- CB25** Sistemul păstrează întregul istoric al fiecărui pacient: fișe generate, diagnostice provizorii, diagnostice confirmate, rețete, recomandări și trimiteri.
- CB26** La o evaluare nouă, istoricul pacientului este consultat automat pentru a adapta fișa generată cu întrebări relevante afecțiunilor anterioare.
- CB27** Sistemul calculează automat un scor de risc pentru fiecare pacient, pe baza vârstei, afecțiunilor cronice și istoricului medical.

3. Normalizarea bazei de date

3.1 Relația universală

Procesul de normalizare pornește de la o singură relație universală care conține toate attributele sistemului:

```
TELEMEDICINA(  
    patient_id, first_name, last_name, birth_date, phone_number,  
    email, password, address, tutor_id,  
    subscription_type, subscription_start_date,  
        subscription_end_date,  
    subscription_cost, subscription_status,  
    payment_paid_at, payment_amount, payment_method,  
    chronic_name, chronic_diagnosed_date,  
    form_created_at, form_status, form_provisional_diagnosis,  
    form_complexity_level,  
    symptom_description, symptom_type,  
    answer_question, answer_answer,  
    doctor_first_name, doctor_last_name, doctor_email,  
    doctor_specialization, doctor_schedule_start,  
        doctor_schedule_end,  
    consultation_scheduled_at, consultation_duration_minutes,  
    consultation_status, consultation_confirmed_diagnosis,  
    consultation_notes, consultation_referral_type,  
    consultation_referral_details, consultation_waiting_list,  
    prescription_issued_at, prescription_medications,  
    prescription_recommendations  
)
```

3.2 Dependente funcționale

Pe baza cerințelor de business identificate, se stabilesc următoarele dependente funcționale:

DF1: `patient_id` $\rightarrow$ `first_name`, `last_name`, `birth_date`, `phone_number`, `email`, `password`, `address`, `tutor_id`

DF2: `email` $\rightarrow$ `patient_id`

DF3: `patient_id` $\rightarrow$ `subscription_type`, `subscription_start_date`, `subscription_end_date`, `subscription_cost`, `subscription_status`

DF4: `subscription_start_date`, `subscription_end_date` $\rightarrow$ `subscription_status`

DF5: `subscription_id` $\rightarrow$ `payment_paid_at`, `payment_amount`, `payment_method`

DF6: `patient_id`, `chronic_name` $\rightarrow$ `chronic_diagnosed_date`

DF7: `form_id` $\rightarrow$ `patient_id`, `form_created_at`, `form_status`, `form_provisional_diagnosis`, `form_complexity_level`

DF8: `form_id` $\rightarrow$ `doctor_id`, `consultation_scheduled_at`, `consultation_duration_minutes`, `consultation_status`, `consultation_confirmed_diagnosis`, `consultation_notes`, `consultation_referral_type`, `consultation_referral_details`, `consultation_waiting_list`

DF9: `doctor_id` $\rightarrow$ `doctor_first_name`, `doctor_last_name`, `doctor_email`, `doctor_specialization`, `doctor_schedule_start`, `doctor_schedule_end`

DF10: `consultation_id` $\rightarrow$ `prescription_issued_at`, `prescription_medications`, `prescription_recommendations`

DF11: `form_id` $\rightarrow$ `answer_question`, `answer_answer`

3.3 Dependente multivaluate

DM1: `patient_id` $\twoheadrightarrow$ `chronic_name`

Un pacient poate avea mai multe afecțiuni cronice, independente între ele.

DM2: `form_id` $\twoheadrightarrow$ `symptom_description`

O fișă medicală conține mai multe simptome, independente între ele.

DM3: `form_id` $\twoheadrightarrow$ `answer_question`

O fișă medicală poate avea mai multe răspunsuri la întrebări, independente între ele.

3.4 Găsirea cheilor candidat

Organizăm attributele după locul în care apar în dependențele funcționale:

Stânga (apar doar în stânga)	Mijloc (apar în ambele părți)	Dreapta (apar doar în dreapta)
patient_id	subscription_id	first_name
form_id	doctor_id	last_name
consultation_id	email	birth_date
		subscription_status
		payment_amount
		chronic_diagnosed_date
		symptom_description
		prescription_medications

Tabela 1: Clasificarea atributelor după apariția în dependențe

Atributele din coloana *Stânga* fac parte cu siguranță din orice cheie candidat. Calculăm închiderea:

```
patient_id+ = {patient_id, first_name, last_name, birth_date,
phone_number, email, password, address, tutor_id,
subscription_type, subscription_start_date,
subscription_end_date, subscription_cost,
subscription_status, payment_paid_at,
payment_amount, payment_method}
```

Nu conține form_id, consultation_id - nu este cheie singur.

$(\text{patient_id}, \text{form_id}, \text{consultation_id})^+ = U$ - aceasta este cheia candidat a relației universale.

Cheia candidată: **(patient_id, form_id, consultation_id)**

Atribute prime: patient_id, form_id, consultation_id

Atribute neprime: toate celelalte attribute din U.

3.5 Forma Normală 1 (1NF)

O schemă de relație este în **1NF** dacă domeniile atributelor sunt indivizibile și fiecare valoare a fiecărui atribut este atomică.

Relația universală **TELEMEDICINA nu este în 1NF** deoarece:

- Un pacient poate avea **mai multe afecțiuni cronice** — atributul **chronic_name** are valori multiple pentru același pacient.
- Un pacient poate avea **mai multe simptome** — atributul **symptom_description** are valori multiple per fișă.

- O fișă poate avea **mai multe răspunsuri** — atributul `answer_question` are valori multiple per fișă.

Aducerea în 1NF: Eliminăm grupurile repetitive și le mutăm în relații separate. După această operație obținem următoarele scheme:

```
T1(patient_id, first_name, last_name, birth_date, phone_number,
    email, password, address, tutor_id,
    subscription_type, subscription_start_date,
    subscription_end_date,
    subscription_cost, subscription_status,
    payment_paid_at, payment_amount, payment_method,
    form_created_at, form_status, form_provisional_diagnosis,
    form_complexity_level,
    doctor_first_name, doctor_last_name, doctor_email,
    doctor_specialization, doctor_schedule_start,
    doctor_schedule_end,
    consultation_scheduled_at, consultation_duration_minutes,
    consultation_status, consultation_confirmed_diagnosis,
    consultation_notes, consultation_referral_type,
    consultation_referral_details, consultation_waiting_list,
    prescription_issued_at, prescription_medications,
    prescription_recommendations)

T2(patient_id, chronic_name, chronic_diagnosed_date)
T3(form_id, symptom_description, symptom_type)
T4(form_id, answer_question, answer_answer)
```

Relațiile **T2**, **T3** și **T4** sunt în 1NF — fiecare atribut este atomic. **T1** este în 1NF dar conține încă redundanțe — va fi adusă în forme normale superioare.

3.6 Forma Normală 2 (2NF)

O schemă de relație în 1NF este în **2NF** dacă orice atribut neprim este **dependent plin** de orice cheie candidat (adică nu depinde de o submulțime proprie a cheii).

Analizăm relația **T1**. Cheia candidată a lui T1 este (`patient_id`, `form_id`, `consultation_id`).

Identificăm dependențele care **nu sunt pline**:

- `patient_id` → `first_name`, `last_name`, `birth_date`, `phone_number`, `email`, `password`, `address`, `tutor_id` — `patient_id` este submulțime a cheii, deci aceste attribute **nu sunt dependente plin** de cheia întreagă. **Violare 2NF.**
- `patient_id` → `subscription_type`, `subscription_start_date`, `subscription_end_date`, `subscription_cost`, `subscription_status` — idem, violare 2NF.
- `form_id` → `form_created_at`, `form_status`, `form_provisional_diagnosis`, `form_complexity_level` — `form_id` este submulțime a cheii. **Violare 2NF.**
- `form_id` → `doctor_id`, `consultation_scheduled_at`, ... — idem, violare 2NF.

- $\text{doctor_id} \rightarrow \text{doctor_first_name}, \dots$ — doctor_id este submulțime tranzitivă. **Violare 2NF.**
- $\text{consultation_id} \rightarrow \text{prescription_issued_at}, \text{prescription_medications}, \text{prescription_recommendations}$ — consultation_id este submulțime a cheii. **Violare 2NF.**

Aducerea în 2NF: Pentru fiecare violare, creăm o relație nouă conținând atributul/atributele determinante și atributele determinate, și eliminăm atributele determinate din T1:

```
R_PAT(patient_id, first_name, last_name, birth_date,
      phone_number,
      email, password, address, tutor_id)
      Cheie: patient_id

R_SUB(patient_id, subscription_type, subscription_start_date,
      subscription_end_date, subscription_cost,
      subscription_status)
      Cheie: patient_id

R_FORM(form_id, patient_id, form_created_at, form_status,
      form_provisional_diagnosis, form_complexity_level)
      Cheie: form_id

R_CONS(form_id, doctor_id, consultation_scheduled_at,
      consultation_duration_minutes, consultation_status,
      consultation_confirmed_diagnosis, consultation_notes,
      consultation_referral_type,
      consultation_referral_details,
      consultation_waiting_list)
      Cheie: form_id

R_DOC(doctor_id, doctor_first_name, doctor_last_name,
      doctor_email,
      doctor_specialization, doctor_schedule_start,
      doctor_schedule_end)
      Cheie: doctor_id

R_PRESC(consultation_id, prescription_issued_at,
      prescription_medications, prescription_recommendations)
      Cheie: consultation_id
```

Toate relațiile obținute, împreună cu **T2**, **T3** și **T4** din pasul 1NF, sunt acum în **2NF** — fiecare atribut neprim depinde plin de cheia sa.

3.7 Forma Normală 3 (3NF)

O schemă în 2NF este în **3NF** dacă niciun atribut neprim nu este **tranzitiv dependent** de nicio cheie.

Analizăm relațiile obținute:

R_PAT: Avem $\text{email} \rightarrow \text{patient_id}$ și $\text{patient_id} \rightarrow \text{first_name}, \dots$. Emailul este cheie candidată, deci nu există dependențe tranzitive. **R_PAT este în 3NF.**

R_SUB: Avem $\text{subscription_start_date}, \text{subscription_end_date} \rightarrow \text{subscription_status}$ (DF4). $\text{subscription_status}$ depinde tranzitiv de patient_id prin intermediul datelor de început/sfârșit. **Violare 3NF.**

Totuși, în implementare păstrăm **status** ca atribut derivat în **SUBSCRIPTION** și îl actualizăm automat printr-un **trigger**, evitând astfel o joncțiune suplimentară la fiecare interogare. Aceasta este o decizie de implementare justificată prin performanță.

R_CONS: doctor_id apare în **R_CONS** ca cheie străină către **R_DOC**. Nu există dependențe tranzitive în interiorul lui **R_CONS**. **R_CONS este în 3NF.**

R_FORM, R_DOC, R_PRESC, T2, T3, T4: Fiecare relație are o singură cheie simplă și toate atributele depind direct de aceasta. **Toate sunt în 3NF.**

Toate relațiile sunt în **3NF**.

3.8 BCNF (Boyce-Codd Normal Form)

O schemă de relație în 1NF este în **BCNF** dacă pentru orice dependență funcțională netrivială $X \rightarrow A \in \Sigma^+$, X este supercheie.

Verificăm fiecare relație:

R_PAT: Dependențele sunt $\text{patient_id} \rightarrow \text{toate atributele}$ și $\text{email} \rightarrow \text{patient_id}$. Ambele stânguri (patient_id și email) sunt superchei (chei candidat). **R_PAT este în BCNF.**

R_SUB: Singura cheie este patient_id . Dependența DF4 ($\text{start_date}, \text{end_date} \rightarrow \text{status}$) are în stânga o mulțime care nu este supercheie. **Violare BCNF.**

Aplicăm algoritmul de descompunere:

- $S_1 = \{\text{start_date}, \text{end_date}, \text{status}\}$
- $S_2 = R_SUB - \{\text{status}\} = \{\text{patient_id}, \text{type}, \text{start_date}, \text{end_date}, \text{cost}\}$

Ca și în cazul 3NF, în implementare păstrăm **status** în aceeași relație și îl gestionăm prin trigger, decizie justificată prin performanță.

R_FORM: Cheia este form_id . Toate dependențele au form_id în stânga. **R_FORM este în BCNF.**

R_CONS: Cheia este form_id (UNIQUE). Toate dependențele non-triviale au form_id în stânga. **R_CONS este în BCNF.**

R_DOC: Avem $\text{doctor_id} \rightarrow \text{toate atributele}$ și $\text{doctor_email} \rightarrow \text{doctor_id}$. Ambele sunt superchei. **R_DOC este în BCNF.**

R_PRESC, T2, T3, T4: Fiecare are o singură cheie simplă și toate dependențele au acea cheie în stânga. **Toate sunt în BCNF.**

După aplicarea algoritmului BCNF, obținem următoarele scheme:

```

R1 = R_PAT(patient_id PK, first_name, last_name, birth_date,
           phone_number, email UNIQUE, password, address,
           tutor_id FK->R1)

R2 = R_SUB(subscription_id PK, patient_id FK->R1, type,
           start_date, end_date, cost, status)

R3 = R_PAY(payment_id PK, subscription_id FK->R2,
           paid_at, amount, method)
-- subscription_id adaugat din relatia de mai sus

R4 = R_FORM(form_id PK, patient_id FK->R1, created_at, status,
           provisional_diagnosis, complexity_level)

R5 = R_CONS(consultation_id PK, form_id FK->R4 UNIQUE,
           doctor_id FK->R6, scheduled_at, duration_minutes,
           status, confirmed_diagnosis, notes,
           referral_type, referral_details, waiting_list)

R6 = R_DOC(doctor_id PK, first_name, last_name,
           email UNIQUE, password, specialization,
           schedule_start, schedule_end)

R7 = R_PRESC(prescription_id PK, consultation_id FK->R5 UNIQUE,
           issued_at, medications, recommendations)

```

3.9 Forma Normală 4 (4NF)

O schemă în BCNF este în **4NF** dacă pentru orice dependență multivaluată netrivială $X \twoheadrightarrow Y$, X este supercheie.

Verificăm dependențele multivaluate identificate:

DM1: $\text{patient_id} \twoheadrightarrow \text{chronic_name}$

În R1, patient_id este supercheie, dar dacă am păstra chronic_name în R1, ar fi o dependență multivaluată netrivială unde stânga este supercheie. Totuși, păstrarea ar genera rânduri duplicate. Aplicăm descompunerea:

```

S1 = {patient_id, chronic_name} -> partial, adaugam
    chronic_diagnosed_date
R8 = CHRONIC_CONDITION(chronic_id PK, patient_id FK->R1,
                       name, diagnosed_date)

```

DM2: $\text{form_id} \twoheadrightarrow \text{symptom_description}$

În T3, form_id nu este supercheie singur (cheia era form_id , $\text{symptom_description}$). Avem dependență multivaluată netrivială unde stânga nu este supercheie. **Violare 4NF**. Aplicăm descompunerea:

```

R9 = SYMPTOM(symptom_id PK, form_id FK->R4,
             description, type)

```

DM3: $\text{form_id} \rightarrow \text{answer_question}$

Similar cu DM2, form_id nu este supercheie în T4. **Violare 4NF.** Aplicăm descompunerea:

```
R10 = FORM_ANSWER(answer_id PK, form_id FK->R4,
                  question, answer)
```

3.10 Concluzie

În urma aplicării algoritmilor 1NF, 2NF, 3NF, BCNF și 4NF, relația universală a fost descompusă în 10 relații, fiecare aflată în forma normală 4NF.

Relație	Tabel implementat	Forma normală obținută
R1	PATIENT	4NF
R2	SUBSCRIPTION	BCNF (status gestionat prin trigger)
R3	SUBSCRIPTION_PAYMENT	4NF
R4	MEDICAL_FORM	4NF
R5	CONSULTATION	4NF
R6	DOCTOR	4NF
R7	PRESCRIPTION	4NF
R8	CHRONIC_CONDITION	4NF
R9	SYMPTOM	4NF
R10	FORM_ANSWER	4NF

Tabela 2: Corespondența dintre relațiile normalizate și tabelele implementate

4. Diagrama Entitate-Asociere

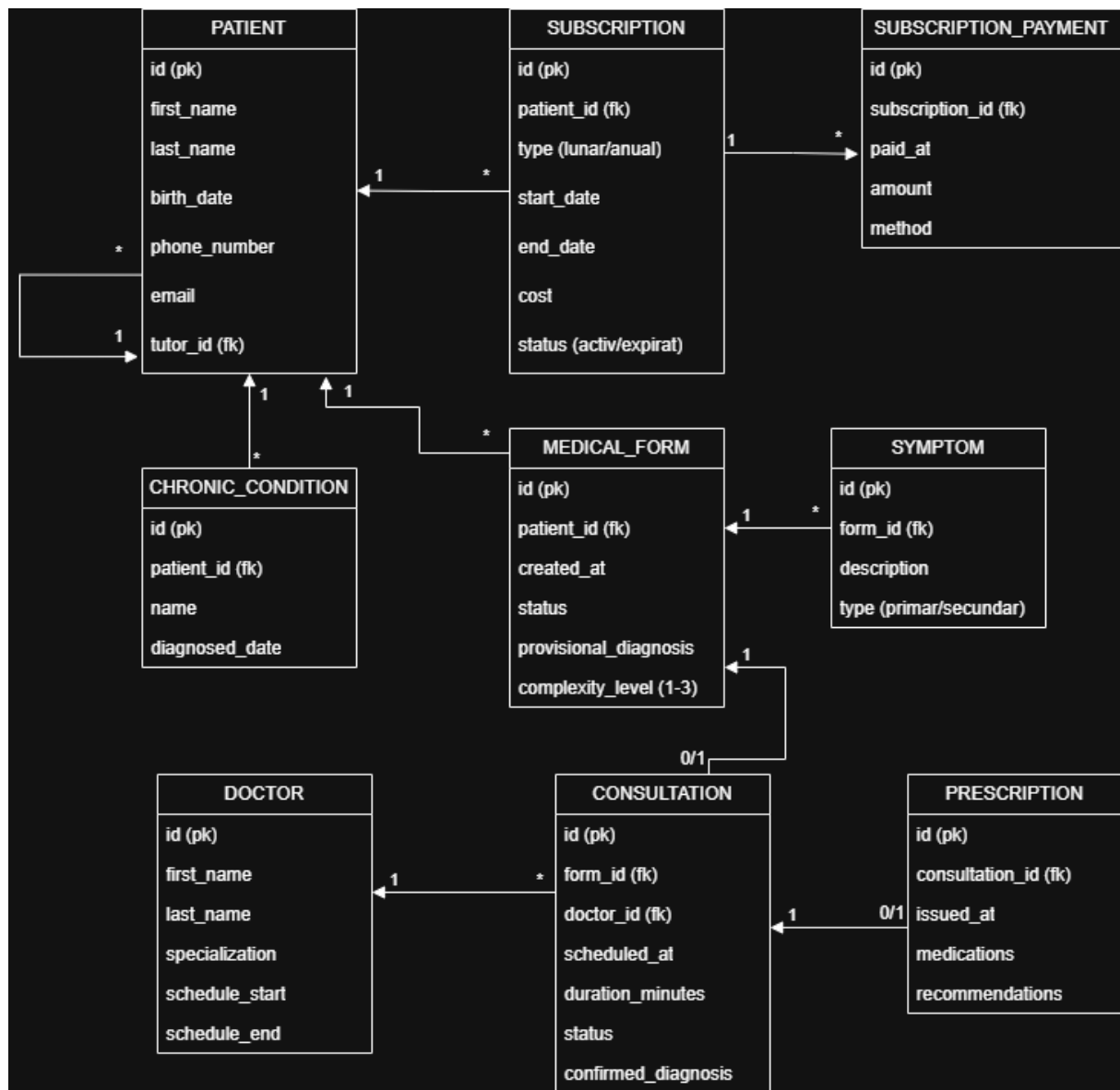


Figura 1: Diagrama E/A a sistemului de telemedicină

4.1 Entități și asocieri

Schema bazei de date conține 10 entități și următoarele asocieri:

PATIENT — SUBSCRIPTION (1:N)

Un pacient poate avea mai multe abonamente în timp (lunar sau anual), dar fiecare abonament aparține unui singur pacient. Relația este implementată prin cheia străină `patient_id` în tabela `SUBSCRIPTION`. S-a ales **ON DELETE CASCADE** deoarece dacă un pacient este șters, abonamentele sale nu mai au relevanță.

SUBSCRIPTION — SUBSCRIPTION_PAYMENT (1:N)

Un abonament poate genera mai multe plăți. S-a ales **ON DELETE CASCADE** deoarece plățile fără abonament sunt lipsite de context.

PATIENT — CHRONIC_CONDITION (1:N)

Un pacient poate avea mai multe afecțiuni cronice înregistrate, independente între ele. Aceasta este justificarea dependenței multivaluate DM1.

PATIENT — PATIENT (relație reflexivă, 1:N)

Un pacient adult poate fi tutorele mai multor pacienți minori. Relația este modelată printr-o cheie străină reflexivă `tutor_id` care referențiază tot **PATIENT(id)**. S-a ales **ON DELETE SET NULL** — dacă tutorele este șters, copiii rămân în sistem cu `tutor_id = NULL`.

PATIENT — MEDICAL_FORM (1:N)

Un pacient poate genera mai multe fișe medicale de-a lungul timpului. Cheia străină `patient_id` din **MEDICAL_FORM** referențiază **PATIENT(id)**.

MEDICAL_FORM — SYMPTOM (1:N)

O fișă medicală conține mai multe simptome (cel puțin 3 primare, plus simptome secundare generate automat). Justifică dependența multivaluată DM2.

MEDICAL_FORM — FORM_ANSWER (1:N)

O fișă medicală poate conține mai multe răspunsuri la întrebările suplimentare generate automat. Justifică dependența multivaluată DM3.

MEDICAL_FORM — CONSULTATION (1:0/1)

O fișă medicală poate genera cel mult o consultație. Cardinalitatea 0/1 reflectă faptul că fișele de complexitate 1 se rezolvă prin rețetă automată. Unicitatea este impusă prin **UNIQUE** pe `form_id` din **CONSULTATION**.

DOCTOR — CONSULTATION (1:N)

Un doctor poate conduce mai multe consultații. S-a ales **ON DELETE RESTRICT** — un doctor nu poate fi șters dacă are consultații asociate, pentru a păstra integritatea istoricului medical.

CONSULTATION — PRESCRIPTION (1:0/1)

O consultație poate genera cel mult o rețetă. Doctorul poate decide și trimiterea la investigații fără a prescrie medicamente. Unicitatea este impusă prin **UNIQUE** pe `consultation_id` din **PRESCRIPTION**.

4.2 Chei primare și decizii de design

Toate tabelele folosesc o cheie primară artificială de tip **INT**, generată automat prin secvențe PostgreSQL (`seq_patient`, `seq_doctor` etc.). Această decizie se justifică prin stabilitate (cheile naturale precum emailul se pot modifica), performanță (join-urile pe **INT** sunt mai rapide decât pe **VARCHAR**) și simplitate în codul aplicației.

4.3 Constrângeri implementate

În schema bazei de date au fost utilizate toate cele 5 tipuri de constrângeri posibile:

Tip	Exemplu în schemă	Justificare
PRIMARY KEY	PATIENT(id), DOCTOR(id)	Identificare unică a fiecărui rând
FOREIGN KEY	SUBSCRIPTION(patient_id)	Integritate referențială între tabele
UNIQUE	PATIENT(email), CONSULTATION(form_id)	Email unic per pacient; o fișă generează cel mult o consultație
NOT NULL	PATIENT(first_name), DOCTOR(specialization)	Câmpurile obligatorii nu pot lipsi
CHECK	type IN ('lunar','anual'), complexity_level BETWEEN 1 AND 3	Validarea valorilor permise la nivel de bază de date

Tabela 3: Tipuri de constrângeri utilizate în schemă

5. Scriptul de creare a tabelelor

5.1 Secțiunile DROP și secvențe

Scriptul începe prin ștergerea tabelelor și secvențelor existente, în ordinea inversă dependențelor pentru a evita erorile de integritate referențială:

```
DROP TABLE IF EXISTS PRESCRIPTION CASCADE;
DROP TABLE IF EXISTS CONSULTATION CASCADE;
DROP TABLE IF EXISTS SYMPTOM CASCADE;
DROP TABLE IF EXISTS MEDICAL_FORM CASCADE;
DROP TABLE IF EXISTS CHRONIC_CONDITION CASCADE;
DROP TABLE IF EXISTS SUBSCRIPTION_PAYMENT CASCADE;
DROP TABLE IF EXISTS SUBSCRIPTION CASCADE;
DROP TABLE IF EXISTS DOCTOR CASCADE;
DROP TABLE IF EXISTS PATIENT CASCADE;
DROP TABLE IF EXISTS FORM_ANSWER CASCADE;
```

Urmează crearea secvențelor pentru auto-incrementarea cheilor primare. Folosirea secvențelor în loc de SERIAL oferă control explicit asupra incrementului și valorii de start:

```
CREATE SEQUENCE seq_patient START 1 INCREMENT 1;
CREATE SEQUENCE seq_subscription START 1 INCREMENT 1;
CREATE SEQUENCE seq_payment START 1 INCREMENT 1;
CREATE SEQUENCE seq_chronic START 1 INCREMENT 1;
CREATE SEQUENCE seq_form START 1 INCREMENT 1;
CREATE SEQUENCE seq_symptom START 1 INCREMENT 1;
CREATE SEQUENCE seq_doctor START 1 INCREMENT 1;
CREATE SEQUENCE seq_consultation START 1 INCREMENT 1;
CREATE SEQUENCE seq_prescription START 1 INCREMENT 1;
CREATE SEQUENCE seq_answer START 1 INCREMENT 1;
```

5.2 Tabela PATIENT

```
CREATE TABLE PATIENT (
    id INT DEFAULT nextval('seq_patient') PRIMARY KEY,
```



```

first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
birth_date DATE NOT NULL,
phone_number VARCHAR(15) NOT NULL,
email VARCHAR(100) NOT NULL UNIQUE,
password VARCHAR(255) NOT NULL,
address TEXT,
tutor_id INT,

CONSTRAINT fk_patient_tutor FOREIGN KEY (tutor_id)
REFERENCES PATIENT(id) ON DELETE SET NULL
);

```

Decizii de design: email are constrângerea UNIQUE deoarece este folosit la autentificare. address este TEXT (fără limită de lungime) deoarece adresele pot varia mult. tutor_id este o cheie străină reflexivă care referențiază tot PATIENT(id), cu ON DELETE SET NULL pentru a nu șterge copiii dacă tutorele este șters.

5.3 Tabela DOCTOR

```

CREATE TABLE DOCTOR (
  id INT DEFAULT nextval('seq_doctor') PRIMARY KEY,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  email VARCHAR(100) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  specialization VARCHAR(100) NOT NULL,
  schedule_start TIME NOT NULL,
  schedule_end TIME NOT NULL,

  CONSTRAINT chk_doctor_schedule CHECK (schedule_end >
    schedule_start)
);

```

Constrângerea CHECK pe program garantează că ora de sfârșit este după ora de început. Algoritmul de scheduling folosește schedule_start și schedule_end pentru a calcula sloturile disponibile.

5.4 Tabela SUBSCRIPTION

```

CREATE TABLE SUBSCRIPTION (
  id INT DEFAULT nextval('seq_subscription') PRIMARY KEY,
  patient_id INT NOT NULL,
  type VARCHAR(10) NOT NULL,
  start_date DATE NOT NULL,
  end_date DATE NOT NULL,
  cost DECIMAL(10,2) NOT NULL,
  status VARCHAR(10) NOT NULL DEFAULT 'activ',

  CONSTRAINT fk_sub_patient FOREIGN KEY (patient_id)
REFERENCES PATIENT(id) ON DELETE CASCADE,

```

```

CONSTRAINT chk_sub_type CHECK (type IN ('lunar', 'anual')),
CONSTRAINT chk_sub_status CHECK (status IN ('activ',
      'expirat')),
CONSTRAINT chk_sub_dates CHECK (end_date > start_date)
);

```

ON DELETE CASCADE asigură că abonamentele sunt șterse automat dacă pacientul este șters. Constrângerea CHECK pe `type` limitează valorile permise la nivel de bază de date, nu doar la nivel de aplicație. Statusul este actualizat automat de un trigger la fiecare INSERT sau UPDATE.

5.5 Tabela MEDICAL_FORM

```

CREATE TABLE MEDICAL_FORM (
  id INT DEFAULT nextval('seq_form') PRIMARY KEY,
  patient_id INT NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT now(),
  status VARCHAR(20) NOT NULL DEFAULT 'nou',
  provisional_diagnosis TEXT,
  complexity_level INT NOT NULL DEFAULT 1,

  CONSTRAINT fk_form_patient FOREIGN KEY (patient_id)
    REFERENCES PATIENT(id) ON DELETE CASCADE,
  CONSTRAINT chk_form_status CHECK (status IN
    ('nou', 'completat', 'programat', 'inchis')),
  CONSTRAINT chk_form_complexity CHECK (complexity_level
    BETWEEN 1 AND 3)
);

```

`provisional_diagnosis` este TEXT (fără limită) deoarece diagnosticul generat automat poate fi lung. Constrângerea pe `complexity_level` garantează că valorile sunt între 1 și 3.

5.6 Tabela CONSULTATION

```

CREATE TABLE CONSULTATION (
  id INT DEFAULT nextval('seq_consultation') PRIMARY KEY,
  form_id INT NOT NULL UNIQUE,
  doctor_id INT NOT NULL,
  scheduled_at TIMESTAMP,
  duration_minutes INT NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'programata',
  confirmed_diagnosis TEXT,
  notes TEXT,
  referral_type VARCHAR(50),
  referral_details TEXT,
  waiting_list BOOLEAN DEFAULT FALSE,

  CONSTRAINT fk_cons_form FOREIGN KEY (form_id)
    REFERENCES MEDICAL_FORM(id) ON DELETE CASCADE,
  CONSTRAINT fk_cons_doctor FOREIGN KEY (doctor_id)

```

```

        REFERENCES DOCTOR(id) ON DELETE RESTRICT,
    CONSTRAINT chk_cons_status CHECK (status IN
        ('programata', 'finalizata', 'anulata')),
    CONSTRAINT chk_cons_duration CHECK (duration_minutes IN
        (10, 20, 30))
);

```

UNIQUE pe form_id implementează relația 1:0/1 — o fișă generează cel mult o consultație. ON DELETE RESTRICT pe doctor_id previne ștergerea accidentală a unui doctor cu consultații în istoric. referral_type și referral_details stochează trimiterile la spital/investigații. waiting_list marchează pacienții în listă de așteptare când nu există sloturi disponibile.

5.7 Tabela FORM_ANSWER și indecși

```

CREATE TABLE FORM_ANSWER (
    id INT DEFAULT nextval('seq_answer') PRIMARY KEY,
    form_id INT NOT NULL,
    question TEXT NOT NULL,
    answer TEXT NOT NULL,

    CONSTRAINT fk_answer_form FOREIGN KEY (form_id)
        REFERENCES MEDICAL_FORM(id) ON DELETE CASCADE
);

CREATE INDEX idx_patient_email ON PATIENT(email);
CREATE INDEX idx_sub_patient ON SUBSCRIPTION(patient_id);
CREATE INDEX idx_form_patient ON MEDICAL_FORM(patient_id);
CREATE INDEX idx_symptom_form ON SYMPTOM(form_id);
CREATE INDEX idx_consultation_doctor ON
    CONSULTATION(doctor_id);
CREATE INDEX idx_consultation_schedule ON
    CONSULTATION(scheduled_at);
CREATE INDEX idx_answer_form ON FORM_ANSWER(form_id);

```

Indecșii au fost creați pe coloanele cel mai frecvent folosite în clauze WHERE și JOIN, optimizând interogările de căutare pacient după email și listarea fișelor unui pacient.

6. Scriptul de populare automată

6.1 Structura generală

Scriptul de populare este implementat ca un bloc anonim PL/pgSQL (DO \$\$) care inserează date realiste și consistente în toate tabelele. Ordinea inserărilor respectă dependențele cheilor străine: DOCTOR și PATIENT întâi (fără FK), apoi SUBSCRIPTION, CHRONIC_CONDITION, MEDICAL_FORM, SYMPTOM, FORM_ANSWER, CONSULTATION, PRESCRIPTION.

6.2 Generarea pacienților

```

FOR v_i IN 1..20 LOOP

```

```

v_email := LOWER(v_prenume || '.' || v_nume);
LOOP
    SELECT COUNT(*) INTO v_email_check FROM PATIENT
    WHERE email = v_email || '@gmail.com';
    EXIT WHEN v_email_check = 0;
    v_email := v_email || FLOOR(RANDOM() * 99 + 1)::TEXT;
END LOOP;
v_email := v_email || '@gmail.com';
v_password := v_prenume || '123';
INSERT INTO PATIENT (first_name, last_name, birth_date,
    phone_number, email, password, address)
VALUES (...);
END LOOP;

```

Loop-ul interior garantează unicitatea emailului — dacă există deja un pacient cu același email, se adaugă un număr aleator până când emailul devine unic. Parola este generată ca prenume123 pentru fiecare utilizator, făcând datele de test ușor de folosit.

6.3 Generarea pacienților minori cu tutori

```

FOR v_i IN 1..5 LOOP
    SELECT id INTO v_tutor_id FROM PATIENT
    ORDER BY id LIMIT 1 OFFSET (v_i % 3);

    v_birth_date := CURRENT_DATE -
        (FLOOR(RANDOM() * 365 * 14) + 365 * 3)::INT;

    INSERT INTO PATIENT (... , tutor_id)
    VALUES (... , v_tutor_id);
END LOOP;

```

OFFSET (v_i % 3) alternază între primii 3 pacienți adulți ca tutori, astfel unii adulți au mai mulți copii. Data de naștere este generată între 3 și 17 ani.

6.4 Generarea abonamentelor cu status corect

```

INSERT INTO SUBSCRIPTION (patient_id, type, start_date,
    end_date,
    cost, status)
VALUES (v_patient_id, v_type, v_start_date, v_end_date,
    CASE v_type WHEN 'lunar' THEN 49.99 ELSE 499.99 END,
    CASE WHEN v_end_date >= CURRENT_DATE THEN 'activ'
    ELSE 'expirat' END);

```

Statusul este calculat dinamic pe baza datei de sfârșit — nu este setat aleator, ci reflectă realitatea datelor generate. Datele de start sunt în trecutul recent (maxim 60 de zile), astfel încât majoritatea abonamentelor să fie active.

6.5 Instrucțiunea RETURNING

```
INSERT INTO MEDICAL_FORM (patient_id, created_at, status,
    provisional_diagnosis, complexity_level)
VALUES (...)
RETURNING id INTO v_form_id;

INSERT INTO SYMPTOM (form_id, description, type)
VALUES (v_form_id, v_symptom1, 'primar');
```

RETURNING id INTO v_form_id captează imediat ID-ul generat de secvență după fiecare INSERT în MEDICAL_FORM. Fără aceasta ar fi necesar un SELECT separat, mai lent și mai fragil. Același ID este folosit imediat la inserările în SYMPTOM și FORM_ANSWER.

7. Prezentarea aplicației

7.1 Arhitectura aplicației

Aplicația client este construită cu **Spring Boot** și **Thymeleaf**, comunicând cu PostgreSQL exclusiv prin **JDBC** pur (fără ORM). Arhitectura urmează modelul MVC:

- **Controller** — primește cererile HTTP, apelează funcțiile PL/pgSQL prin `JdbcTemplate` și trimite datele către template-uri
- **Template HTML** — pagini Thymeleaf care afiează datele primite
- **PostgreSQL** — toată logica de business este la nivel de server

7.2 Autentificare și înregistrare

Aplicația are două tipuri de utilizatori: pacienți și medici. Autentificarea se face prin email și parolă, iar sesiunea este menținută cu `HttpSession`. Pacienții noi se pot înregistra prin pagina `/register`.

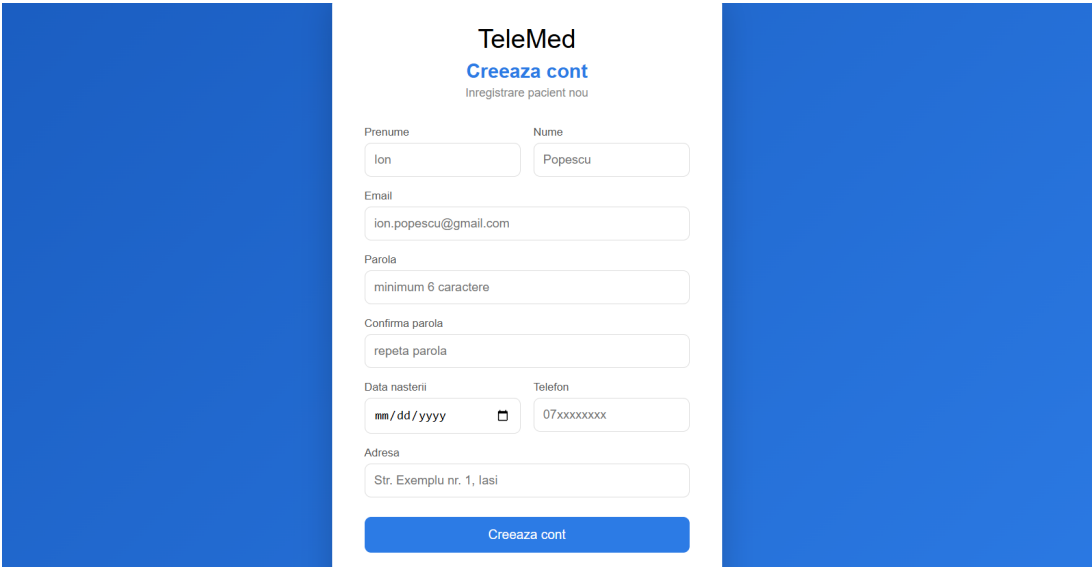


Figura 2: Printscreen pagina de înregistrare pacient

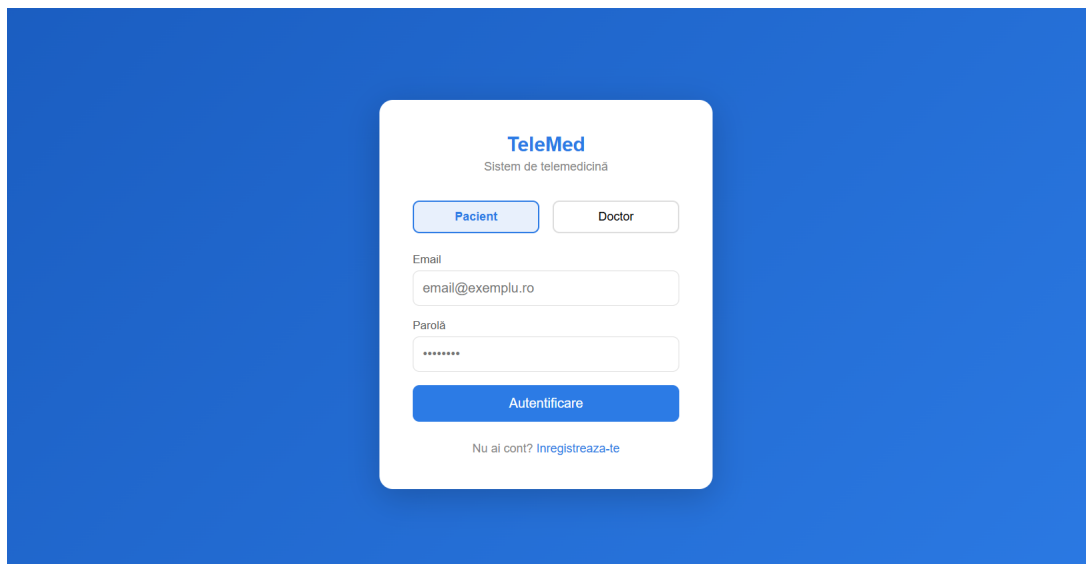


Figura 3: Printscreen pagina de login pacient

7.3 Dashboard pacient

După autentificare, pacientul vede un dashboard personalizat care include statusul abonamentului său, o alertă dacă abonamentul expiră în mai puțin de 7 zile, scorul de risc calculat automat pe baza afecțiunilor cronice și istoricului, și ultimele 5 fișe medicale.

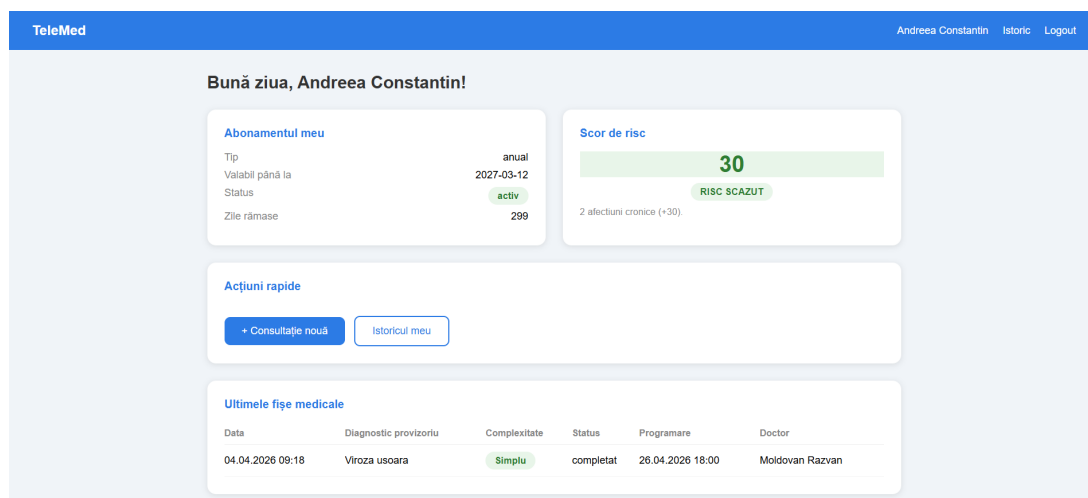


Figura 4: Printscreen dashboard pacient

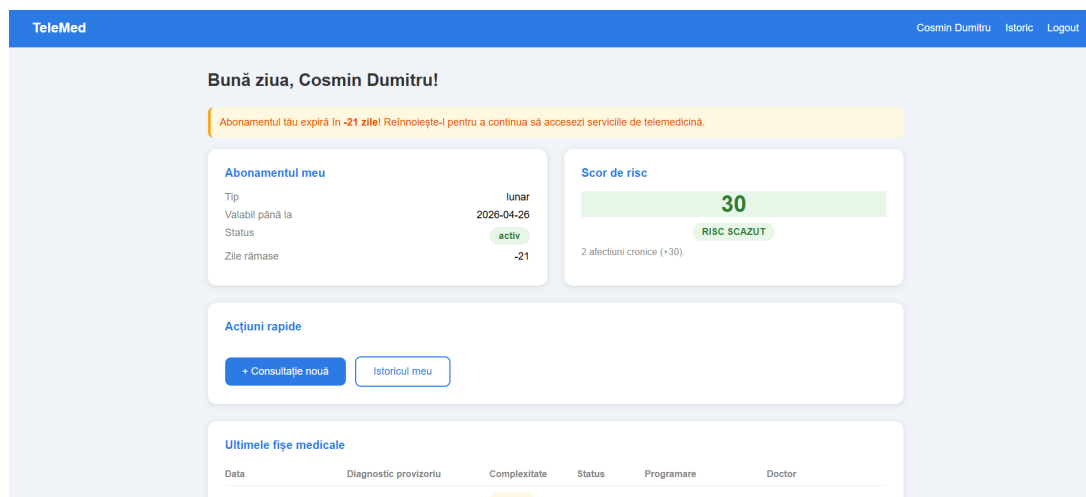


Figura 5: Printscreen dashboard pacient caruia urmeaza să îi expire abonamentul

7.4 Fluxul de consultatie

Fluxul de consultație este implementat în 3 pași:

Pasul 1 — Introducere simptome: Pacientul introduce 3 simptome principale. Sistemul apelează procedura `genereaza_fisa_medicala` care analizează simptomele, vârsta și afecțiunile cronice.

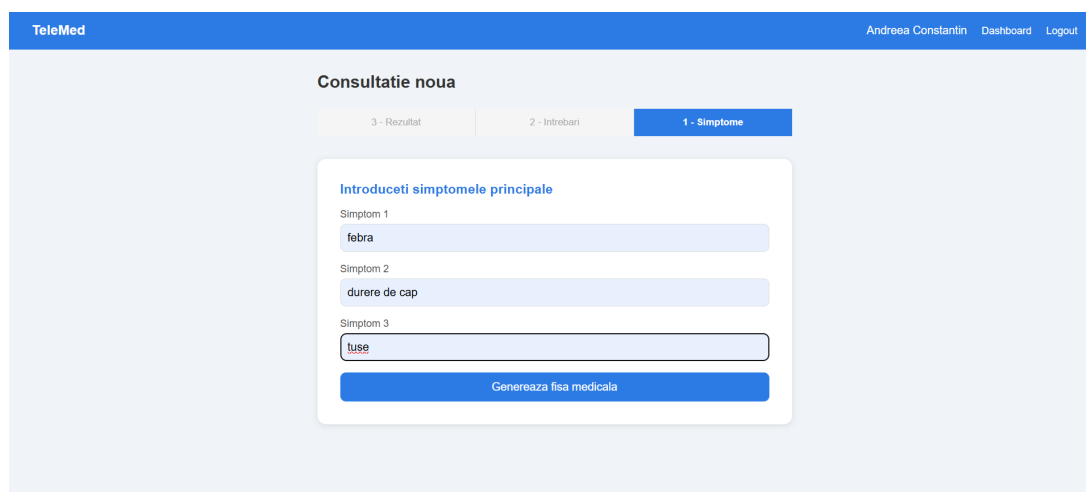


Figura 6: Printscreen pasul 1

Pasul 2 — Întrebări suplimentare: Se afiează diagnosticul provizoriu, nivelul de complexitate și întrebările generate automat. Pentru cazurile urgente (complexitate 3) apare un banner roșu cu recomandarea de a merge la urgențe. Dacă pacientul are simptome similare cu istoricul, apare un avertisment galben.

TeleMed Andreea Constantin Dashboard Logout

Consultatie noua

3 - Rezultat 2 - Intrebări 1 - Simptome

Fisa medicala generata

Diagnostic provizoriu:
Posibila gripa sezoniera - atentie sporita datorita afectiunilor cronice: Hipertensiune arteriala, Anxietate

Complexitate: Mediu

Raspundeti la urmatoarele intrebari

Aveti febra peste 39 grade? (da/nu)
Raspunsul tau

Ati luat medicamentele pentru afectiunile cronice astazi? (da/nu)
Raspunsul tau

Simptomele s-au agravat in ultimele 24h? (da/nu)
Raspunsul tau

Figura 7: Printscreen pasul 2 - caz mediu

TeleMed Bianca Popa Dashboard Logout

Consultatie noua

3 - Rezultat 2 - Intrebări 1 - Simptome

Fisa medicala generata

Diagnostic provizoriu:
Simptome generale - evaluare necesara tinand cont de afectiunile cronice: Astm bronsic

Complexitate: Mediu

Atentie - simptome similare cu istoricul tau:
ATENȚIE: Simptom similar cu un caz de urgenta din istoricul dumneavoastră - diagnostic anterior: Afectiune ce necesita investigatii suplimentare (2026-03-26 09:18:27.776162)
ATENȚIE: Simptom similar cu un caz de urgenta din istoricul dumneavoastră - diagnostic anterior: Afectiune ce necesita investigatii suplimentare (2026-03-26 09:18:27.776162)

Raspundeti la urmatoarele intrebari

De cate zile aveti simptomele?
Raspunsul tau

Figura 8: Printscreen pasul 2 - atentionare în funcție de simptome anterioare

Pasul 3 — Rezultat: Pentru complexitate 1 se generează automat o rețetă cu medicamente fără prescripție. Pentru complexitate 2-3 se apelează algoritmul de scheduling și se afiează doctorul și data programării.

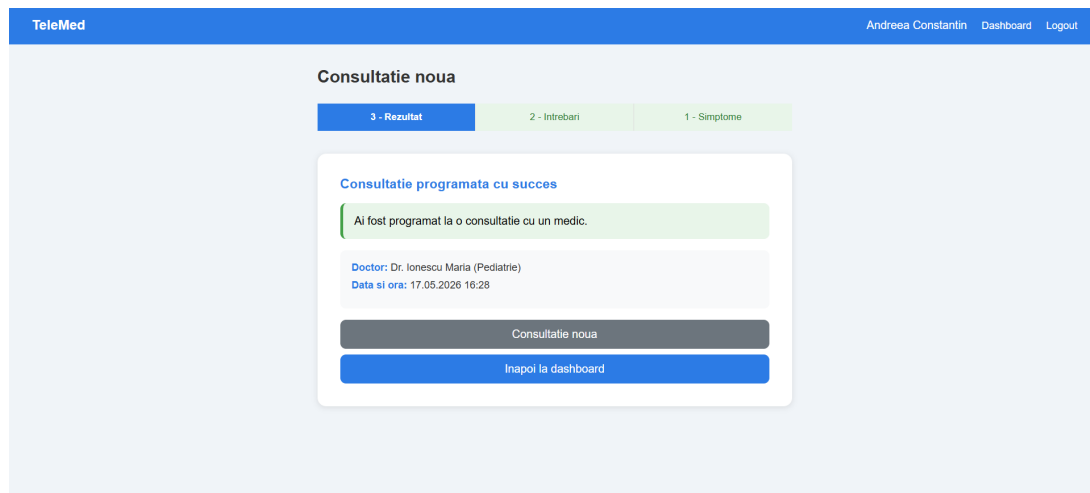


Figura 9: Printscreen pasul 3 - programare consultatie

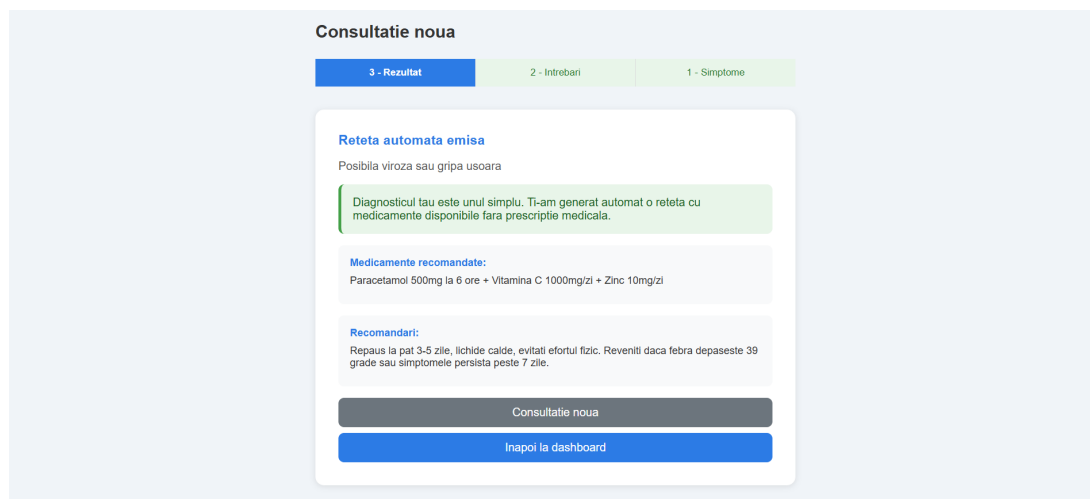


Figura 10: Printscreen pasul 2 - diagnostic si recomanări

7.5 Algoritmul de scheduling

Acesta este cel mai complex algoritm implementat în PL/pgSQL. Procedura `scheduleaza_consultatie` găsește primul slot liber astfel:

```
WHILE v_day_counter < v_max_days AND NOT v_found LOOP
  FOR v_doc IN SELECT id, schedule_start, schedule_end
                FROM DOCTOR ORDER BY id
  LOOP
    v_slot_start := v_day + v_doc.schedule_start;

    FOR v_occupied IN
      SELECT scheduled_at,
             scheduled_at + (duration_minutes || '
             minutes')::INTERVAL
      FROM CONSULTATION
      WHERE doctor_id = v_doc.id
             AND scheduled_at::DATE = v_current_day
```

```

        AND status != 'anulata'
        AND waiting_list = FALSE
    ORDER BY scheduled_at
LOOP
    IF v_slot_end <= v_occupied.scheduled_at THEN
        v_found := TRUE;
    ELSE
        v_slot_start := v_occupied.ends_at;
    END IF;
END LOOP;
END LOOP;
v_current_day := v_current_day + 1;
END LOOP;

```

Algoritmul iterează zi cu zi (maxim 14 zile), și pentru fiecare zi iterează prin toți doctorii. Pentru fiecare doctor, găsește primul gol între consultațiile deja programate care încap consultația nouă. Dacă nu se găsește niciun slot în 14 zile, pacientul este adăugat în lista de așteptare (`waiting_list = TRUE`).

7.6 Triggere

Trigger 1 — Actualizare automată status abonament:

```

CREATE OR REPLACE FUNCTION trg_update_subscription_status()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.end_date < CURRENT_DATE THEN
        NEW.status := 'expirat';
    ELSE
        NEW.status := 'activ';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_subscription_status
BEFORE INSERT OR UPDATE ON SUBSCRIPTION
FOR EACH ROW
EXECUTE FUNCTION trg_update_subscription_status();

```

Acest trigger se declanșează la orice INSERT sau UPDATE pe SUBSCRIPTION și calculează automat statusul pe baza datei de expirare. Astfel, statusul este întotdeauna consistent cu datele, indiferent de cum este inserat sau modificat abonamentul.

Trigger 2 — Închidere automată fișă medicală:

```

CREATE OR REPLACE FUNCTION trg_close_form_on_consultation()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.status = 'finalizata' AND OLD.status !=
        'finalizata' THEN
        UPDATE MEDICAL_FORM SET status = 'inchis'

```

```

        WHERE id = NEW.form_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_consultation_finalizata
AFTER UPDATE ON CONSULTATION
FOR EACH ROW
EXECUTE FUNCTION trg_close_form_on_consultation();

```

Acest trigger se declanșează după ce o consultație este marcată ca **finalizata** și actualizează automat statusul fișei medicale asociate la **inchis**. Condiția `OLD.status != 'finalizata'` previne execuții repetate.

7.7 Excepții custom prinse în client

Aplicația client prinde excepțiile aruncate de PL/pgSQL și afiează mesaje prietenoase utilizatorului:

```

} catch (Exception exception) {
    String message = exception.getMessage();
    if (message != null &&
        message.contains("ABONAMENT_INACTIV")) {
        redirectAttributes.addFlashAttribute("error",
            "Nu ai un abonament activ.");
    } else if (message != null &&
        message.contains("PACIENT_INEXISTENT")) {
        redirectAttributes.addFlashAttribute("error",
            "Contul tau nu a fost gasit in sistem.");
    } else if (message != null &&
        message.contains("SIMPTOME_INSUFICIENTE")) {
        redirectAttributes.addFlashAttribute("error",
            "Introduceti cele 3 simptome obligatorii.");
    }
}
}

```

Excepțiile implementate sunt: P0001 (abonament inactiv), P0002 (pacient inexistent), P0003 (simptome insuficiente), P0004 (fișă inexistentă), P0005 (consultație deja existentă), P0007 (complexitate invalidă pentru rețetă automată), P0008 (doctor inexistent).

7.8 Dashboard doctor și rapoarte

Medicul vede în dashboard-ul său consultațiile programate pentru ziua curentă, cu detalii complete despre fiecare pacient (vârstă, afecțiuni cronice, simptome). Poate finaliza consultația, prescrie o rețetă și poate decide trimiterea pacientului la spital, investigații sau specialist.

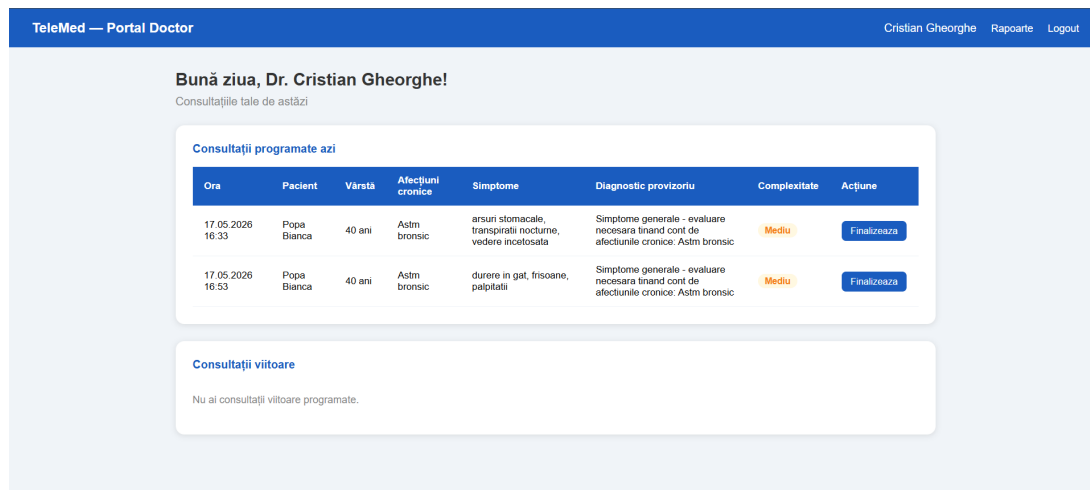


Figura 11: Printscreen dashboard doctor

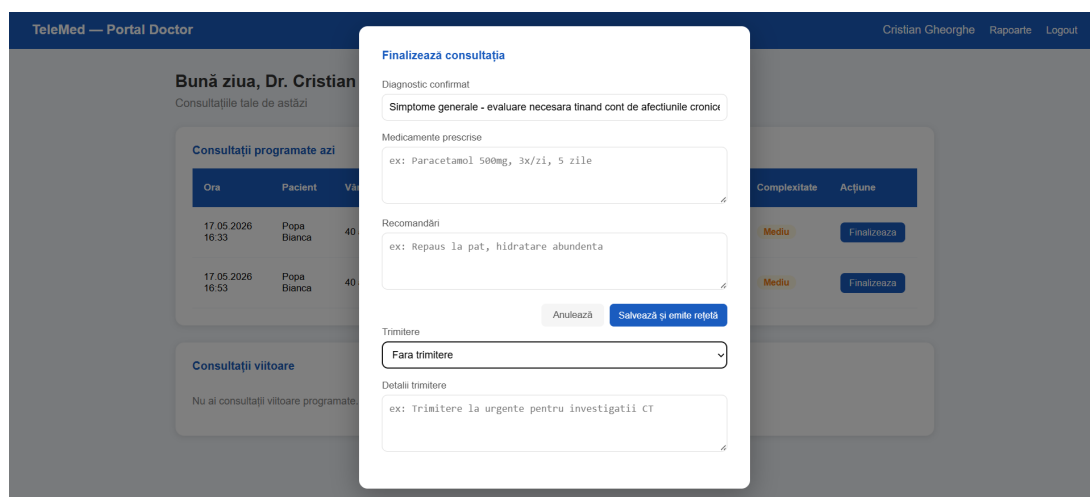


Figura 12: Printscreen modal finalizare consultatie

Pagina de rapoarte oferă statistici complexe generate de funcția `get_raport_statistic()`: număr de consultații per doctor cu rata de finalizare, diagnostice cele mai frecvente, pacienții cu cele mai multe fișe, timp mediu până la programare și distribuția pe niveluri de complexitate. De asemenea, sunt afișate alertele pentru pacienții al căror abonament expiră în următoarele 7 zile.

TeleMed - Portal Doctor			Cristian Gheorghe	Dashboard	Logout
Rapoarte si statistici					
- Statistici generale					
Categorie	Detaliu	Valoare			
Consultatii per doctor	Gheorghe Cristian (Cardiologie)	7 consultatii (0.0% finalizate)			
Consultatii per doctor	Ionescu Maria (Pediatrie)	6 consultatii (50.0% finalizate)			
Consultatii per doctor	Constantin Elena (Neurologie)	5 consultatii (40.0% finalizate)			
Consultatii per doctor	Marin Tudor (Cardiologie)	4 consultatii (50.0% finalizate)			
Consultatii per doctor	Rusu Bogdan (Dermatologie)	4 consultatii (0.0% finalizate)			
Consultatii per doctor	Popescu Andrei (Medicina generala)	4 consultatii (50.0% finalizate)			
Consultatii per doctor	Stoica Raluca (Pediatrie)	2 consultatii (100.0% finalizate)			
Consultatii per doctor	Serban Gabriela (Oncologie)	1 consultatii (100.0% finalizate)			
Consultatii per doctor	Oprea Catalin (Dermatologie)	1 consultatii (0.0% finalizate)			
Consultatii per doctor	Moldovan Razvan (Ortopedie)	1 consultatii (0.0% finalizate)			
Consultatii per doctor	Lungu Andreea (Gastroenterologie)	1 consultatii (0.0% finalizate)			

Figura 13: Printscreen pagina rapoarte

Consultatii per doctor	Viad Simona (Neurologie)	
Consultatii per doctor	Matei Ioana (Gastroenterologie)	
Consultatii per doctor	Dinu Mihai (Medicina generala)	
Diagnostic frecvent	Afectiune ce necesita investigatii suplimentare	10 cazuri
Diagnostic frecvent	Gripa sezoniera	9 cazuri
Diagnostic frecvent	Viroza usoara	6 cazuri
Diagnostic frecvent	Posibila gripa sezoniera - atentie sporita datorita afectiunilor cronice: Hipotroidism	4 cazuri
Diagnostic frecvent	Posibila gripa sezoniera - atentie sporita datorita afectiunilor cronice: Artrita reumatoida	4 cazuri
Pacient cu cele mai multe fise	Nistor Paul	6 fise
Pacient cu cele mai multe fise	Chiriac Lucian	5 fise
Pacient cu cele mai multe fise	Constantin Andreea	5 fise
Timp mediu pana la programare	Toate cazurile	357.5 ore
Abonamente active	Total	25 abonamente
Complexitate cazuri	Nivel 1	8 fise (17.8%)
Complexitate cazuri	Nivel 2	27 fise (60.0%)
Complexitate cazuri	Nivel 3	10 fise (22.2%)

Figura 14: Printscreen pagina rapoarte

7.9 Istoricul pacientului

Funcția `get_istoric_pacient(patient_id)` returnează într-un singur apel întregul istoric medical al pacientului: fișe, simptome, diagnostice provizorii și confirmate, rețete și trimiteri. Aceste informații sunt reutilizate la evaluari noi prin funcția `detecteaza_simptome_similare`.

TeleMed

Bianca PopaDashboardConsultație nouăLogout

Istoricul meu medical

Afecțiuni cronice

Afecțiune	Data diagnosticului
Astm bronic	2024-02-09

Abonamente

Tip	Valabil până la	Cost	Status
lunar	2027-04-17	49.99 RON	activ

Fise medicale si consultatii

Data	Simptome	Diagnostic provizoriu	Complexitate	Status	Doctor	Programare	Diagnostic confirmat	Medicamente	Trimitere	Detalii trimitere
2026-05-17 16:32:40.800673	frisoane, durere in gat, palpitații	Simptome generale - evaluare necesara tinand cont de afecțiunile cronice: Astm	Mediu	completat	Gheorghe Cristian	2026-05-17 16:53:00.0	-	-	-	-

Figura 15: Printscreen pagina istoric pacient

7.10 Funcționalități avansate implementate

Pe lângă cerințele de bază, sistemul implementează o serie de funcționalități avansate la nivel de server de baze de date:

Scoring risc pacient — Funcția `calculeaza_scor_risc(patient_id)` calculează automat un scor de risc între 0 și 100 pentru fiecare pacient, pe baza vârstei (vârștnicii și sugarii primesc punctaj mai mare), numărului de afecțiuni cronice (+15 puncte per afecțiune) și istoricului de cazuri urgente (+20 puncte per caz de complexitate 3). Rezultatul este clasificat în *Risc scăzut*, *Risc mediu* sau *Risc ridicat* și este afișat în dashboard-ul fiecărui pacient.

Detectare simptome similare cu istoricul — Funcția `detecteaza_simptome_similare(patient_id, symptom1, symptom2, symptom3)` compară simptomele introduse cu istoricul medical al pacientului. Dacă se găsesc simptome similare cu consultații anterioare, pacientul este avertizat în pasul 2 al fluxului de consultație, cu menționarea diagnosticului anterior și a nivelului de urgență al aceluia caz.

Listă de așteptare automată — Dacă algoritmul de scheduling nu găsește niciun slot disponibil în următoarele 14 zile, pacientul este adăugat automat într-o listă de așteptare (`waiting_list = TRUE` în tabela `CONSULTATION`), fără a arunca o excepție. Pacientul primește un mesaj informativ în interfață.

Categorii de vârstă detaliate — Procedura de generare a fișei medicale clasifică pacienții în 5 categorii de vârstă (sugar, copil, adolescent, adult, vârstnic), fiecare cu logica proprie de diagnostic. De exemplu, sugarii primesc automat complexitate 3 și sunt direcționați către consultație urgentă, iar vârstnicii cu afecțiuni cronice primesc complexitate 3 și evaluare prioritară.

Trimitere la spital/investigatii/specialist — Doctorul poate marca o consultație cu o trimitere de tip *spital*, *investigatii* sau *specialist*, cu detalii aferente. Aceste informații sunt salvate în tabela `CONSULTATION` și sunt vizibile în istoricul medical al pacientului.

Raport statistic complex — Funcția `get_raport_statistic()` generează statistici agregate din mai multe surse: număr de consultații per doctor cu rata de finalizare,

diagnostice cele mai frecvente, pacienții cu cele mai multe fișe, timpul mediu între crearea fișei și programarea consultației, număr de abonamente active și distribuția cazurilor pe niveluri de complexitate.

Alertare abonamente expirare — Funcția `get_abonamente_expira_curand()` returnează lista pacienților al căror abonament expiră în următoarele 7 zile, cu datele lor de contact. Rezultatele sunt afișate atât în dashboard-ul pacientului (banner de avertizare) cât și în pagina de rapoarte a medicului.

Raport zilnic doctor — Funcția `get_raport_zilnic_doctor(doctor_id)` returnează toate consultațiile programate pentru ziua curentă ale unui doctor, împreună cu detalii complete despre fiecare pacient: vârstă, afecțiuni cronice, simptome primare și diagnostic provizoriu, facilitând pregătirea medicului pentru fiecare consultație.